

bandit0

ssh – log in

ls – list files and directories

cat – display contents of a text file

password - 6y2kwnwK6grgvwvpvLaa2T1cpFEKOhNR

exit – leave the account

dash file name: use ./- or -- -

password - PK8fYLZg2hnHSz83plBL1iEPKdD3QToB

spaces in file names: use quotation marks or backslash

--spaces in this filename-- = ./"—spaces in this filename—"

password - 7ZZ2LFrykP2zEyvBl4m3clL7tGYJPME

file: identifies file types

du: monitors storage

find: locates files

password - xzTXq1rDJQVVAzdv5cHq1TQytTWufAMq

```
bandit4@bandit:~/inhere$ file ./*
./-file00: data
./-file01: data
./-file02: data
./-file03: data
./-file04: data
./-file05: data
./-file06: data
./-file07: ASCII text
./-file08: data
./-file09: data
```

ASCII text is the only human readable type

Password - 6C7h9GD8M6ai5nr7wo1RonrzFjj9ylrG

find . -type f -size 1033c ! -executable

-executable: checks if its executable

- **find**: Invokes the file search utility.
- **.**: Specifies the starting directory (the current directory and all subdirectories).
- **-type f**: Restricts the search results exclusively to regular files, omitting directories.
- **-size 1033c**: Filters for the exact size. The **c** suffix is critical; it explicitly instructs the utility to count in individual **bytes** rather than the default 512-byte blocks

Password - pXa26xhMWaC2SvDotA4r9EgZkulOeSBW

```
find / -type f -user bandit7 -group bandit6 -size 33c  
2>/dev/null
```

2>/dev/null: hides all the "Permission denied" errors generated when scanning directories you aren't authorized to read.

Grep: search for specific text patterns inside files or standard input streams

```
grep -i "error" server.log
```

password - Bmnnvf82KzQlfxgAl2d1zYbr1u9pr3E3

```
grep "millionth" data.txt
```

man: manual for commands

password: VR1ljMayciFxbnUokuQmJFw6QC9Vktub

```
sort data.txt | uniq -u
```

piping: | allows multiple commands at once

password: EjmOSvuAu7sGAHqHVcBDPirRe9T03kxl

strings data.txt | grep "="

strings: finds strings in a file

password: B0s2khmbT9u0geKuOoVGW3JZKhndE3BG

Base64 is a binary-to-text encoding scheme. It can often be recognised by equal signs at the end of the data. However, this is not always the case. Linux has a command called **base64** that allows for encoding and decoding in Base64. For decoding, we need to use the flag **-d**.

Base64 -d data.txt

Password: pYfOY6HwUsDj5rL9UvyhU7MCmv8vN5Ro

tr: translate

tar: bundles multiple files and directories into a single .tar

gzip: shrinks individual files to reduce size disk space to a .gz extension

bzip2: achieves smaller files sizes than gzip but longer to process

xxd: hex dump/decode: used to inspect or edit raw file data/

```
tr 'A-Za-z' 'N-ZA-Mn-za-m'
```

cat data.txt | tr 'A-Za-z' 'N-ZA-Mn-za-m'

for rot13

```
>
  Save output to a file.
>>
  Append output to a file.
<
  Read input from a file.
2>
  Redirect error messages.
|
  Send the output from one program as input to another program.

Streams
Every program you may run on the command line has 3 streams, STDIN,
STDOUT and STDERR.
```

```
tr 'A-Za-z' 'N-ZA-Mn-za-m' <<< "Pack My Box With Five Dozen
Liquor Jugs"
```

password: GROozWPO8QyN0mGrjUkID0WCYkZiQxrN

mkdir: creates a new folder

cp: copies a file/folder

mv: moves or renames a file/folder

```
xxd -r hexdump_data compressed_data
1 bandit12@bandit:~$ cd /tmp
2 bandit12@bandit:/tmp$ mkdir -p tmp
3 /tmp/tmp.W5t1vua6G9
4 bandit12@bandit:/tmp$ cd /tmp/tmp.W5t1vua6G9
5 bandit12@bandit:/tmp/tmp.W5t1vua6G9$ cp ~/data.txt .
6 bandit12@bandit:/tmp/tmp.W5t1vua6G9$ ls
7 data.txt
8 bandit12@bandit:/tmp/tmp.W5t1vua6G9$ mv data.txt hexdump_data
9 bandit12@bandit:/tmp/tmp.W5t1vua6G9$ ls
10 hexdump_data
```

Decompress it:

- If it's **gzip**: `gunzip data.gz`
- If it's **bzip2**: `bunzip2 data.bz2`
- If it's **tar**: `tar -xf data.tar`

Password: qQYQiHOBPR8zR61qxYqX45quvihF2uzk

SCP is a command that uses SSH to transfer data over the network. The syntax to get a file from a remote host looks like the following: `scp -P <port> <user>@<IP>:<remotefilepath> <localfilepath>`. To send a file to a remote host, the local file path needs to stand at the beginning.

```
ssh bandit14@bandit.labs.overthewire.org -p2220 -i sshkey.private
```

```
scp bandit13@bandit.labs.overthewire.org:sshkey.private . -P2220
```

When you see a command like `chmod 755 myfile.sh`, you are setting permissions using math for **Owner / Group / Others**:

- **7** (4+2+1) = Owner can Read, Write, and Execute.
- **5** (4+0+1) = Group can Read and Execute (no editing).
- **5** (4+0+1) = Others can Read and Execute (no editing).

If `chmod` changes permissions *after* a file is already created, `umask` is the **automatic filter** that sets permissions the exact moment a file is born.

`nc` stands for **Netcat**, and it is often called the "Swiss Army Knife" of networking. It allows you to send and receive data across a network between two computers.

Imagine it like a digital walkie-talkie. You can set one computer to "listen" on a specific channel (called a **port**), and use another computer to "talk" to it.

A Quick Example:

If you want to send a quick text message from Computer A to Computer B over your home Wi-Fi:

1. **On Computer B (The Listener):** You tell it to listen on port 9999.

```
Bash
```

```
nc -l 9999
```

2. **On Computer A (The Sender):** You connect to Computer B's IP address on that same port.

```
Bash
```

```
nc 192.168.1.50 9999
```

Now, anything you type into Computer A's terminal will instantly pop up on Computer B's screen. Network engineers use this to test if routers are blocking ports, or to quickly move data between servers.

Password: aaWecNkG4FhxJQxz07uiwzVP6bJiYS65

nc localhost 30000

Password: pbLYuZtTg4MgaqfJx8jbA9gKKGqM68A7 .

```
openssl s_client -connect localhost:30001
```

openssl: encrypted

nc: not

they do the same thing however

password: kS0Hf0u5HiXFwKMKFqXvPdOTNGGa0X8V

nmap [Scan Type] [Options] {Target Specification}

- **-sS (TCP SYN Scan):** The default "half-open" scan. It is fast and stealthy.
- **-sT (TCP Connect Scan):** Completes the full connection. Used if you don't have administrative privileges on your machine.
- **-sU (UDP Scan):** Scans for UDP ports (like DNS, DHCP, or NTP) instead of TCP.
- **Single port:** -p 80
- **Multiple ports:** -p 80,443,8080
- **Range of ports:** -p 1-100
- **Every single port (1 to 65535):** -p-

- **-sV (Version Detection):** Probes open ports to determine service name and version info.
- **-O (OS Detection):** Enables operating system fingerprinting.
- **-A (Aggressive Scan):** A powerful combination shortcut. It enables OS detection (-O), version detection (-sV), script scanning (-sC), and traceroute all at once.

However, openssl s_client has built-in interactive commands. If you type certain words at the very beginning of a line, the OpenSSL tool intercepts them instead of sending them to the server:

- Typing **R** triggers a **RENEGOTIATING** state.
- Typing **Q** closes the connection.
- Typing **K** triggers a **KEYUPDATE**.

Solution: -ign_eof

```
cat /etc/bandit_pass/bandit16 | openssl s_client -connect localhost:31790 -quiet also works
```

The /tmp directory is a shared space designed for programs and users to store temporary files. Because it's meant for quick, temporary tasks, the system is configured to give **everyone** permission to read and write files there.

Method 2: Using the cat command (Quickest for small text)

If you already have the text copied to your clipboard and just want to dump it into a new file without opening an editor:

1. **Run the command:** Type `cat >` followed by your filename.

Bash

```
cat > filename.txt
```

2. **Paste your info:** Paste your text into the terminal.
3. **Save and exit:** Press **Enter** to move to a new line, then press **Ctrl + D**. This tells Linux you are done writing.

nano is a beginner-friendly, in-terminal text editor.

1. **Open or create the file:** Type `nano` followed by the name you want to give the file.

Bash

```
nano filename.txt
```

2. **Paste your info:** * If you are using a standard terminal, just right-click and select **Paste**, or use the shortcut **Ctrl + Shift + V** (on Linux/Windows) or **Cmd + V** (on Mac).
3. **Save and exit:** * Press **Ctrl + O** then hit **Enter** to save (write out) the file.
 - o Press **Ctrl + X** to exit the editor.

The **-i** flag in SSH stands for "**Identity File**" and is used to specify a private key file for public key authentication when logging into a remote server.

Password: pWXMAZoxGC8JmDMfmT5MGESobMM3vnj2

```
diff file1.txt file2.txt
```

Understanding the Output Symbols

When you run a basic `diff`, the output uses specific symbols to tell you how to turn the first file into the second file:

- **a (Add):** Text needs to be added to the first file.
- **c (Change):** Text in the first file needs to be modified.
- **d (Delete):** Text needs to be removed from the first file.
- **< (Less-than):** Indicates lines that are in the *first* file but missing from the second.
- **> (Greater-than):** Indicates lines that are in the *second* file but missing from the first.

```
bandit17@bandit:~$ ls
```

```
passwords.new passwords.old
```

```
bandit17@bandit:~$ diff passwords.new passwords.old
```

```
42c42
```

```
< OQxXZjELndr90zuhOTDYBEoml0SZITXI
```

> qOg5pVOjPx9x9VccyYBADiT4xxyoUB8D

- **42c42:** This means line 42 in the first file (passwords.new) needs to be **changed** to match line 42 in the second file (passwords.old).
- **< 0QxXZjELndr90zuhOTDYBEomI0SZITXI:** The < symbol points to the first file. This is the password currently inside passwords.new.
- **—:** This is just a divider between the two files' contents.
- **> qOg5pV0jPx9x9VccyYBADiT4xxyoUB8D:** The > symbol points to the second file. This is the password inside passwords.old.

Password: 0QxXZjELndr90zuhOTDYBEomI0SZITXI

Not only does ssh allow log ins to machine but also remote execution of commands by adding the commands after the ssh expression

```
ssh bandit18@bandit.labs.overthewire.org -p2220 cat  
readme
```

```
password: KpsOfPkcP7i1FlIExk2QEjyt6dw8dxZI
```

setuid: program runs with permissions of file's owner
(ex.changing passwords)

Plaintext

```
- rwx r-x r-- 1 alice dev-team 1024 July 5 2026 report.txt
  \_/ \_/ \_/      ^      ^
  |   |   |        |      |--> Group Owner
  |   |   |        |--> User Owner
  |   |   |        |--> Others permissions (read-only: r--)
  |   |-----> Group permissions (read and execute: r-x)
  |-----> User (Owner) permissions (full access: rwx)
```

-rwsr-x--- 1 bandit20 bandit19 14880 Jun 24 14:58

bandit20-d

Means owner is bandit20, group is bandit19,

'-rwsr-x-' means the user bandit19 can execute the binary, but the binary is executed as user bandit20.

./bandit20-do cat /etc/bandit_pass/bandit20

Password: 4pljcunZ0fK2vmp3lwG8Vf7VhxD6pOA

Tmux or screen: split screen for command prompts

. Set Up the Terminal Pane

Split your `tmux` window so you can run the server and client simultaneously: [\[1, 2\]](#)

- Split horizontally: press `Ctrl + b` then `"`
- Or vertically: press `Ctrl + b` then `%`
- Switch between panes: press `Ctrl + b` then an Arrow Key [\[1, 2\]](#)

Can do it either using `&`(work it background) or `tmux`(two separate windows)

Bash: opens up a new command line

&: command working background

CTRL-Z: pauses the program

Jobs: list all programs paused or running background

echo -n "4pljcunZ0fK2vmp3lwG8Vf7VhxD6pOA" | nc -l
-p1234

Bg: resumes a paused program keeps it background

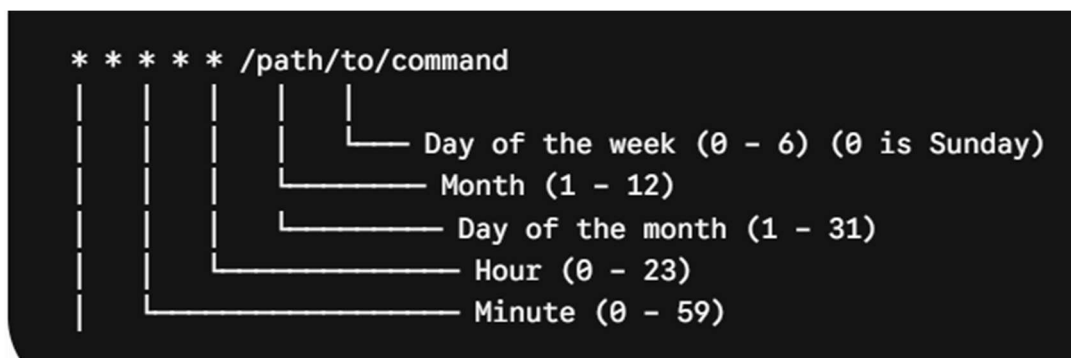
Fg: foreground

-n: no new lines

Piped password to send it instantly when the binary connects

Once you have saved your newly obtained password, you can kill the active `tmux` window and all running tasks inside it by pressing `Ctrl + b` then `&`. Type `y` to confirm. [[2](#)]

Password: bW9kBv5WC3P4yoDyf12LSdGuNz5ka6hY



chmod

644

/tmp/t7O6lds9S0RqQh9aMcz6ShpAoZKF7fgv

cat /etc/bandit_pass/bandit22 >

/tmp/t7O6lds9S0RqQh9aMcz6ShpAoZKF7fgv

password: RYVux2rHEm9tiXHmLFzuR7Vhx6AZQMEz

```
bandit22@bandit:/etc/cron.d$ cat cronjob_bandit23
@reboot bandit23 /usr/bin/cronjob_bandit23.sh &> /dev/null
* * * * * bandit23 /usr/bin/cronjob_bandit23.sh &> /dev/null
bandit22@bandit:/etc/cron.d$ cat /usr/bin/cronjob_bandit23.sh
#!/bin/bash

myname=$(whoami)
mytarget=$(echo I am user $myname | md5sum | cut -d ' ' -f 1)

echo "Copying passwordfile /etc/bandit_pass/$myname to /tmp/$mytarget"

cat /etc/bandit_pass/$myname > /tmp/$mytarget
bandit22@bandit:/etc/cron.d$ cd
bandit22@bandit:~$ whoami
bandit22
bandit22@bandit:~$ cat /etc/bandit_pass/bandit22
RYVux2rHEm9tiXHmLFzuR7Vhx6AZQMEz
bandit22@bandit:~$ cat /etc/bandit_pass/bandit23
cat: /etc/bandit_pass/bandit23: Permission denied
bandit22@bandit:~$ echo I am user bandit23 | md5sum | cut -d ' ' -f 1
8ca319486bfbbc3663ea0fbe81326349
bandit22@bandit:~$ cat /tmp/8ca319486bfbbc3663ea0fbe81326349
gKXDTAXnIz3OBxiPjRZ2uqutUIPZrBsw
```

password: gKXDTAXnIz3OBxiPjRZ2uqutUIPZrBsw

touch: create empty files.

first line in a shell script is always: `#!/bin/bash`

Run a script every single minute:

```
* * * * * /scripts/monitor.sh
```

Run a backup every day at 2:30 AM:

```
30 2 * * * /scripts/backup.sh
```

Run a cleanup script every Sunday at midnight:

```
0 0 * * 0 /scripts/cleanup.sh
```

Run a task every 15 minutes: You can use the `/` operator to specify intervals.

```
*/15 * * * * /scripts/check-status.sh
```

```
#!/bin/bash
```

```
cat /etc/bandit_pass/bandit24 > /tmp/cmon/password
```

Password: hVQMk3IJNsmQ7VF3ubyrNNBom7BOgVXv

In the world of computing, a **daemon** (pronounced "demon" or "day-mon") is a background process that runs continuously and handles system requests without any direct intervention from a user.

Nano: allows creating a shell script and write code: `ctrl + o` to save and then `ctrl + x` to exit

```
#!/bin/bash
```

```
for i in {0000..9999}
```

```
do
```

```
    echo hVQMk3IJNsmQ7VF3ubyrNNBom7BOgVXv $i >> tryouts.txt
```

done

```
cat tryouts.txt | nc -N localhost 30002 > results.txt
```

To run it: `./brute.sh`

`rm` deletes files

`grep -v "Wrong" results.txt`

`-v`: ones w/o this word/phrase

`tmp/25`

password: SoHfqMOEqIX2IYKVciZxvgpR9a2Djx4P

The `/etc/passwd` file is a plain-text database in Unix-like operating systems that stores essential user account information. It contains one record per line for each system user or service account, and defines details like user IDs (UID), home directories, and login shells

```
Vim is a text editor. It enables you to run shell commands as well. It is possible to use vim to break out of a restricted environment and spawn a shell. To spawn the user's default shell, the command :shell is used. To change the shell to '/bin/bash' the command is :set shell=/bin/bash. And then :shell.
```

Log in with shrinked screen System forces you into more.

Force more to pause → Press `v` to jump into vim.

Inside vim → Tell it to use a real shell (`:set shell=/bin/sh`).

Spawn the shell (:shell) → You are free. You now have a normal command prompt.

:qa! To leave vim

bandit26@bandit:~\$ cat /etc/bandit_pass/bandit26

option1(shorter): just type :e /etc/bandit_pass/bandit26 in vim

option2(longer): type :set shell=/bin/bash and then :shell,
then type in cat /etc/bandit_pass/bandit26

password: jHdv2ELQhT22BkprMNDjybZDAkw1zeBJ

rwsr

**s implies setuid: ./bandit27-do + any command permissible
bandit27**

./bandit27-do cat /etc/bandit_pass/bandit27

Vim is a highly advanced text editor, and it has features that allow it to interact directly with the underlying operating system.

Password: STJLJBRRphMxKB392CT4iOr5CbzPU9ER

Use `git clone <url>` to download an existing repository from a remote server (like GitHub) onto your local machine. [[1](#), [2](#), [3](#), [4](#), [5](#)]

git clone ssh://username@hostname:port/path/to/repository.git

ssh://bandit27-git@bandit.labs.overthewire.org:2220/home/bandit27-git/repo

windows: dir linux: ls

windows: type linux: cat

password: y8Yd2ssKcpHpud7UvOSOxwamRMzIGIeQ

```
C:\Users\golfh\repo\repo>type README.md
```

```
# Bandit Notes
```

```
Some notes for level29 of bandit.
```

```
## credentials
```

```
- username: bandit29
```

```
- password: xxxxxxxxxx
```

```
C:\Users\golfh\repo\repo>git log
```

```
commit 83d77407b76c9f86ac4e691a47618641c9d527ba (HEAD -> master, origin/master, origin/HEAD)
```

```
Author: Morla Porla <morla@overthewire.org>
```

```
Date: Wed Jun 24 14:59:06 2026 +0000
```

```
fix info leak
```

```
commit 13bbc4d2414ffe0439b8ee4f5e5c2949780cf4b3
```

```
Author: Morla Porla <morla@overthewire.org>
```

```
Date: Wed Jun 24 14:59:06 2026 +0000
```

```
add missing data
```

```
commit f3334fbccbf9446a6af88a3c71021c2f57163322
```

```
Author: Ben Dover <noone@overthewire.org>
```

```
Date: Wed Jun 24 14:59:06 2026 +0000
```

```
initial commit of README.md
```

```
C:\Users\golfh\repo\repo>git show 83d77407b76c9f86ac4e691a47618641c9d527ba
commit 83d77407b76c9f86ac4e691a47618641c9d527ba (HEAD -> master, origin/master, origin/HEAD)
Author: Morla Porla <morla@overthewire.org>
Date: Wed Jun 24 14:59:06 2026 +0000
```

```
fix info leak
```

```
diff --git a/README.md b/README.md
index 42331d9..5c6457b 100644
--- a/README.md
+++ b/README.md
@@ -4,5 +4,5 @@ Some notes for level29 of bandit.
## credentials
```

```
- username: bandit29
-- password: Em7eGtqaMySwNFjCpwzzHhLhospOcdt0
+- password: xxxxxxxxxx
```

password: Em7eGtqaMySwNFjCpwzzHhLhospOcdt0

- git branch: List (-a), create, or delete branches
- git checkout <branch_name>/git switch <branch_name>: Switch branches
- git merge: Join two or more branches

password: jq9Dfg2rXsfYsWMgFuKlXhphjdH7UsgX

git tag: labels to specific commits (product versions for example)

```
C:\Users\golfh\tmp2\repo>git tag
secret
```

```
C:\Users\golfh\tmp2\repo>git show secret
```

82Nkymb1pGBYmIXG6ZQ8YldBYstHpfUf

Password: 82Nkymb1pGBYmIXG6ZQ8YldBYstHpfUf

Unlike Linux/Bash, Windows Command Prompt includes the literal single quotes ' and the trailing space before the > inside the text file.

Because of that command, your key.txt file actually contains: 'May I come in?' (with quotes and a trailing space)

But the server expects exactly: May I come in? (no quotes, no extra spaces)

Order of operation:

```
Git add -f(force/ignores .gitignore)key.txt
```

```
Git commit -m 'commitmessage'
```

```
Git push origin master
```

In vim type i then type the commit message at the top then press esc then type :wq to save and exit.

Password: pWuj5jBQ6IgV0NXwiH6g1pXRF8S1YvbT

What the `sh` Command Does

- **Command Interpreter:** It reads inputs from your keyboard or a script file and runs them.
- **System Standard:** It is the legacy standard shell present on all Unix/Linux systems.
- **Modern Symlink:** On modern Linux systems, `sh` is usually a symbolic link pointing to `bash` or `dash`.
- **Execution Flag:** Using `sh -c "command"` executes a specific command string directly inside a new shell process.
 - `>> ls`
 - `sh: 1: LS: not found`
 - Under the hood, the restricted API is passing your input into `sh` like this: `sh -c "YOUR_INPUT"`. Since it capitalizes text, entering `ls` results in `sh -c "LS"`, which fails.

To bypass the uppercase restriction, you must use a command or variable that contains no lowercase letters, like `$0\`

In Linux, **\$0** is a special built-in shell parameter that stores the name of the shell or the shell script currently being executed.

Executing `$0` will trick the `sh` loop into spawning a new unrestricted `bash` shell

Password: u4P2CyPOwPGL94RdD9Uo2FxFwvnFswM